

UNIT- III

Syntax Analysis (Part-II)

Bottom Up Parsers :

⇒ Constructing of a parse tree for any given input string beginning at bottom and working towards root is called as Bottom Up parsing.

⇒ Bottom up parsers uses Right most derivation in reverse to Construct parse tree.

ex:- $S \rightarrow aAcBe$

$A \rightarrow Ab|b$

$B \rightarrow d$

input : abcde

-for parse table Construction as RMD in reverse order:

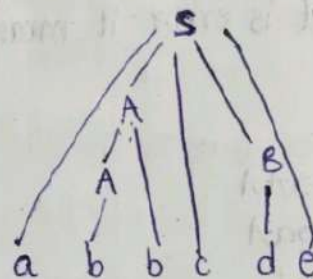
RMD:

$S \rightarrow aAcBe$

$\rightarrow aAcde$

$\rightarrow aAbcde$

$\rightarrow abcde$



1) Shift Reduce Parser (S-R) :-

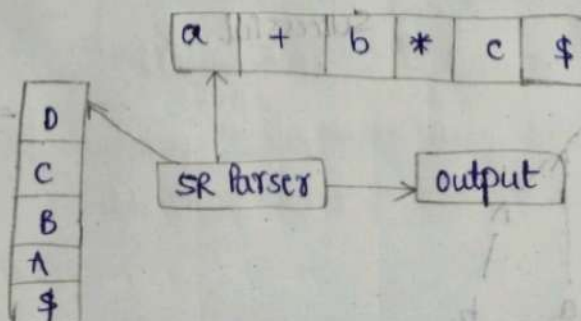
SR parsing usually depends upon the concept of handle pruning inorder to take reduce action.

-Handle Pruning: The process of Replacing the RHS with LHS. Handle is R.H.s of some production in sentential form.

⇒ The concept of SR parsing usually takes the help of a stack, while performing SR parsing. The following points are to be considered.

i) we first need to locate the substring to be reduced in the right sentential form.

ii) we have to decide which production is to be chosen, if there are more than one production with that substring in the right hand side.



Initial Configuration is:

stack

\$

input

w\$

4 Actions of Shift Reduce Parser:

- 1) Shift: Push each terminal symbol from the input string on to stack.
- 2) Reduce: If a handle is found on top of the stack then Replace the handle with its L.H.S.
- 3) Accept: If stack contains \$s and the input contains only \$ then SR parser derives the input string Successfully.
- 4) Error: If parser neither performs shift, reduce nor Accept then it is error it means unsuccessful parsing.

Example: 1

$S \rightarrow AA$

$A \rightarrow aA$

$A \rightarrow b$

input: $w = abab\$$

stack

input

Action

\$

abab\$

shift a

\$a

bab\$

shift b

\$ab

ab\$

reduce $A \rightarrow b$

\$aA

ab\$

reduce $A \rightarrow aA$

\$A

ab\$

shift a

\$Aa

b\$

shift b

\$Aab

\$

reduce $A \rightarrow b$

\$AaA

\$

reduce $A \rightarrow aA$

\$AA

\$

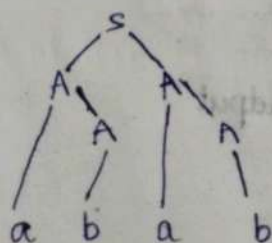
reduce $S \rightarrow AA$

\$S

\$

Successful.

Parse Tree:



⇒ shift reduce parser is possible for some of the ambiguous grammar.

Example : 2

$S \rightarrow S + S$

$S \rightarrow S - S$

$S \rightarrow (S)$

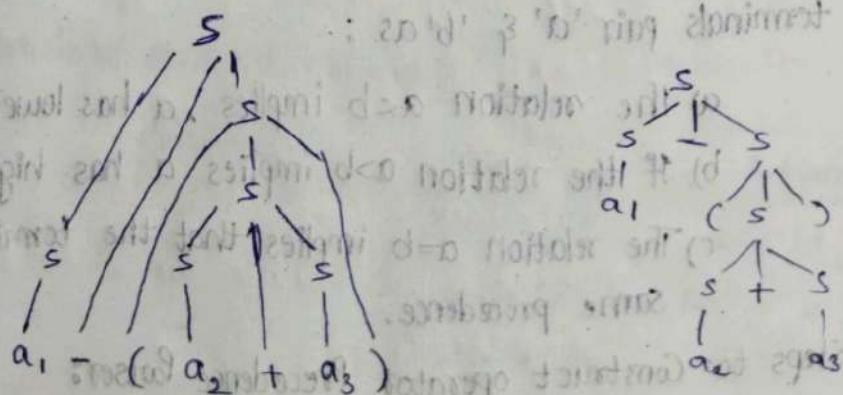
$S \rightarrow a$

input: $w = a_1 - (a_2 + a_3) \$$

Sol:-

stack	input	Action
\$	$a_1 - (a_2 + a_3) \$$	shift a_1
$\$ a_1$	$- (a_2 + a_3) \$$	Reduce $s \rightarrow a_1$
$\$ S$	$- (a_2 + a_3) \$$	shift -
$\$ S -$	$(a_2 + a_3) \$$	shift (
$\$ S - ($	$a_2 + a_3) \$$	shift a_2
$\$ S - (a_2$	$+ a_3) \$$	Reduce $s \rightarrow a_2$
$\$ S - (s$	$+ a_3) \$$	shift +
$\$ S - (s +$	$a_3) \$$	shift a_3
$\$ S - (s + a_3$	) \$	Reduce $s \rightarrow a_3$
$\$ S - (s + s$	) \$	shift)
$\$ S - (s + s)$	\$	Reduce $s \rightarrow s + s$
$\$ S - (s)$	\$	Reduce $s \rightarrow (s)$
$\$ S - s$	\$	Reduce $s \rightarrow s - s$
$\$ S$	\$	Successful

Parse tree:



Operator Precedence Parser:

⇒ It is a kind of shift reduce parsing method that can be applied to a small class of operator grammars and it can be ambiguous grammar also.

⇒ An operator grammar has two important characteristics.

i) No ϵ -productions

ii) No two terminals are adjacent in production rules.

Ex:-

$S \rightarrow AaB$ ✓, $S \rightarrow AabB$ ✓, $S \rightarrow aab$

$S \rightarrow AaBC$ ✗, $S \rightarrow aA$ }
 $A \rightarrow b|c$ } ✗

Ex:- $E \rightarrow EAE | (E) | -E | id$

$A \rightarrow + | - | * | /$ It is not an operator precedence grammar

if we substitute A in E :

$E \rightarrow E+E | E-E | E * E | E / E | E \wedge E | (E) | -E | id$

This is operator precedence grammar.

This parser consists of:

- 1) An input string to be parsed followed by a \$ which indicates end of input string.
- 2) A stack containing sequence of grammar symbols with a \$ at the bottom of stack.
- 3) Three kinds of precedence relations will exist between the pair of terminals pair 'a' & 'b' as:
 - a) The relation $a < b$ implies, a has lower precedence than b.
 - b) If the relation $a > b$ implies a has high precedence than b.
 - c) The relation $a = b$ implies that the terminals 'a' & 'b' has same precedence.

Steps to Construct operator Precedence Parser:

Step 1: check whether the given grammar is operator grammar or not. If not convert it into operator grammar.

Step 2: Construct relational table

⇒ Rules to construct relational table:

- Identifiers has highest precedence, \$ has least precedence.
- we can't compare two identifiers such as $id+id$ so we use dash (-)
- For same operators we choose precedence & associativity.
- If \$ & \$ occurred then parsing is successful.

Step 3: Parse the string

Step 4: Generate a parse tree

Ex: $E \rightarrow EAE \mid id$
 $A \rightarrow + \mid *$ parse the string $id+id*id$.

Sol: Step 1: Given grammar is not operator grammar. Convert it by substituting A into E .

$E \rightarrow E+E \mid E * E \mid id$

$id > + > *$ (precedence)

Step 2: relational table:

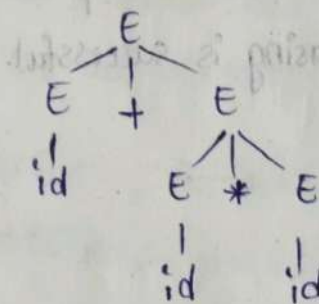
	id	+	*	\$
id	-	>	>	>
+	<	>	<	>
*	<	>	>	>
\$	<	<	<	accept

Step 3: parse the input string $id+id*id\$$

Stack	Relation	input	Action
\$	<	$id+id*id\$$	shift id
$\$id$	>	$+id*id\$$	Reduce $E \rightarrow id$
$\$E$	<	$+id*id\$$	shift +
$\$E+$	<	$id*id\$$	shift id
$\$E+id$	>	$*id\$$	Reduce $E \rightarrow id$
$\$E+E$	<	$*id\$$	shift *
$\$E+E*$	<	$id\$$	shift id
$\$E+E*id$	>	$\$$	Reduce $E \rightarrow id$

Stack	Relation	Input	Action
$\$ E * E * E$	$>$	$\$$	Reduce $E \rightarrow E * E$
$\$ E + E$	$>$	$\$$	Reduce $E \rightarrow E + E$
$\$ E$	$\wedge$	$\$$	Accept

Step 4 : Parse tree is :



LR(k) parsers :

⇒ It is a bottom up parsing technique that can be used to parse large class of CFGs.

⇒ In LR(k)

'L' stands for left to right scanning of the input to construct parse tree.

'R' stands for right most derivation in reverse.

'k' indicates no. of look ahead symbols to make a parsing decision.

⇒ LR-family :

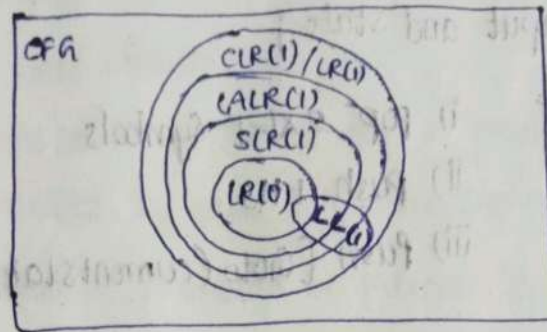
- 1) LR(0) parser
- 2) SLR(1) Parser
- 3) LALR(1) Parser
- 4) CLR(1) Parser

⇒ Ambiguous grammar is not suitable for LR(k) parser.

⇒ CLR(1) is more powerful.

$$LR(0) \leq SLR(1) \leq LALR(1) \leq CLR(1)$$

⇒ Venn diagram representing LR(K) family:



⇒ Every LL(1) is also LR(1) but need not be viceversa.

⇒ LR(0) and SLR(1) uses LR(0) items.

⇒ LALR(1) & CLR(1) uses LR(1) items.

LR(K) Parser Structure:

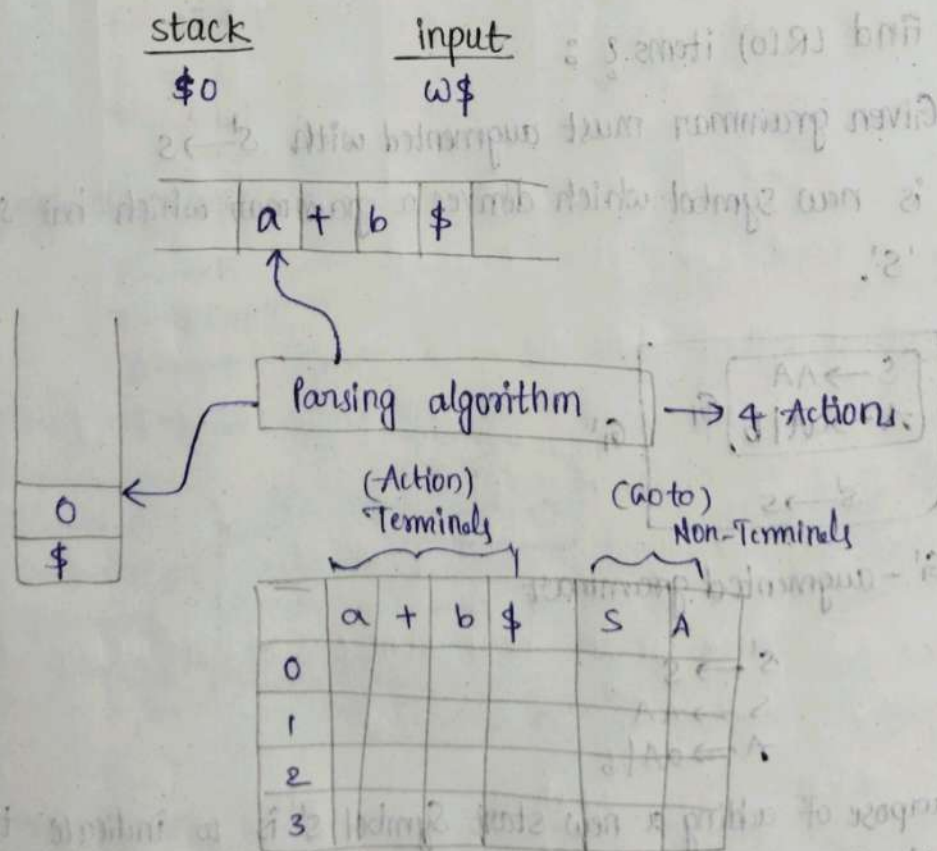
⇒ LR(K) family uses the same parser structure, same parsing algorithm.

⇒ It has one input buffer, in buffer each cell store one input symbol.

Whatever the input string, that is stored in buffer ends with \$ symbol to indicate end of input string.

⇒ It uses one stack to hold symbols, initially which contains '\$0'.

⇒ initial Configuration is:



4 Actions:-

- 1) shift j - Push input and state j
- 2) Reduce - $A \rightarrow \alpha$
 - i) pop $|\alpha|$ symbols
 - ii) push LHS
 - iii) push [Goto (current state, non-terminal)]
- 3) Accept - Successful
- 4) Error - Unsuccessful parsing

LR(0) Item:

$\Rightarrow$ If $A \rightarrow xyz$ Generate 4 items:

$$A \rightarrow \cdot xyz$$

$$A \rightarrow x \cdot yz$$

$$A \rightarrow xy \cdot z$$

$$A \rightarrow xyz \cdot$$

$\Rightarrow$ In $A \rightarrow x \cdot yz$ left of dot means derivation completed (some part of product), right of dot ($\cdot$) means expecting to derived.

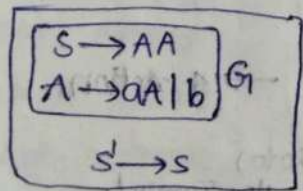
$\Rightarrow$ If $A \rightarrow \epsilon$ it has one item $A \rightarrow \cdot$.

How to find LR(0) items?

Step 1: Given grammar must augmented with $s' \rightarrow s$

s' is new symbol which derives a grammar which has start symbol 'S'.

Ex:-



$$s' \rightarrow s$$

G_1' - augmented grammar

$$s' \rightarrow s$$

$$S \rightarrow AA$$

$$A \rightarrow aA | b$$

$\Rightarrow$ The purpose of adding a new start symbol s' is to indicate the parser about when it should stop the parsing process & announce the acceptance of given input string.

$\Rightarrow S' \rightarrow \cdot S$ indicates (start state) derivation with S is to be start.
 $\Rightarrow S' \rightarrow S \cdot$ indicates (end state) derivation with S is over.

Step 2: Find the closure(I)

If I represents a set of items for a grammar G then closure of I can be constructed by using the following rules.

Rule 1: Every item of I should be present in closure(I)

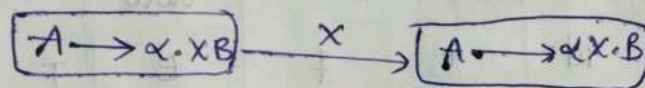
Rule 2: If there is a production $A \rightarrow \alpha \cdot B$ where B is non terminal & assuming $B \rightarrow \gamma$ is a production rule that we have to add $B \rightarrow \cdot \gamma$ in the closure of I (if it is not already present).

Step 3: Goto

This is represented as $\text{Goto}(I, x) = J$

I - Set of LR(0) items

x - It can be a terminal or non-terminal symbol



Here x is terminal or non-terminal.

4. Ex:- Construct LR(0) items for the following grammar

$E \rightarrow E + T$

$E \rightarrow T$

$T \rightarrow id$

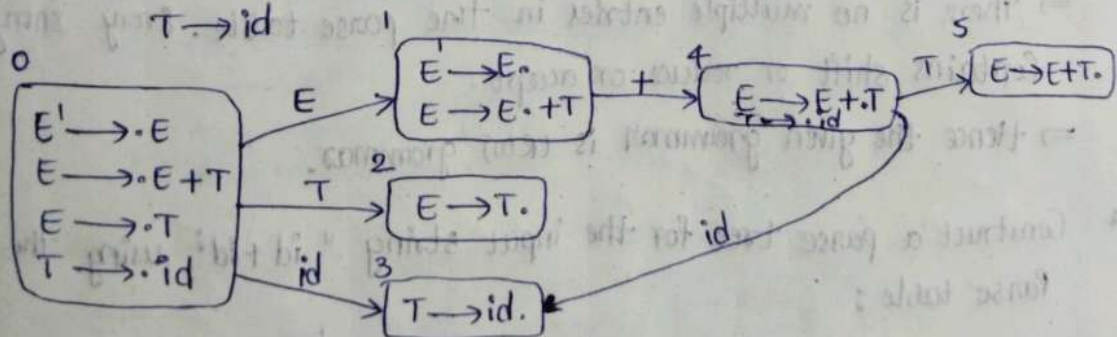
Sol:- Augmented grammar:

$E' \rightarrow E$

$E \rightarrow E + T$

$E \rightarrow T$

$T \rightarrow id$



$\Rightarrow 6$ states

Construction of LR(0) parsing table :

Rule 1 : If $S' \rightarrow s$ is in "I" then

Action(I, \$) = Accept

Rule 2 : If $A \rightarrow \alpha \cdot a \beta$ then

Action(I, a) = shift j

Rule 3 : If $A \rightarrow \alpha \cdot B \beta$ then

Goto(I, B) = j

Rule 4 : If $A \rightarrow \alpha \cdot$ then

Action[I, all terminal] = Reduce with $A \rightarrow \alpha$

* Parsing table for the Example 4 :-

	Action			Goto	
	id	+	\$	E	T
0	S3			1	2
1		S4	Accept		
2	$E \rightarrow T$	$E \rightarrow T$	$E \rightarrow T$		
3	$T \rightarrow id$	$T \rightarrow id$	$T \rightarrow id$		
4	S3				5
5	$E \rightarrow E+T$	$E \rightarrow E+T$	$E \rightarrow E+T$		

⇒ Here all undefined entries are error entries.

⇒ There is no multiple entries in the parse table. Every entry contains shift or reduce or accept.

⇒ Hence the given grammar is LR(0) grammar.

* Construct a parse tree for the input string "id + id" using the LR(0) Parse table :

stack
\$0
\$0id3

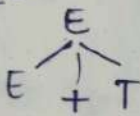
input
id + id \$
+ id \$

Action
shift id & state 3
 $T \rightarrow id$ pop 2 symbols & reduce.
Push T & state change

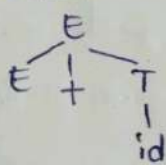
stack	input	Action
\$OT2	+id\$	$E \rightarrow T$ pop 2 Symbols, push E, state change (OE)
\$OE1	+id\$	shift + & 4
\$OE1+4	id\$	shift id & 3
\$OE1+4id3	\$	$T \rightarrow id$; pop id, 3, push T, state change (OT2)
\$OE1+4T5	\$	$E \rightarrow E+T$; pop 6 Symbols, push E, state change (OE)
\$OE1	\$	Accept

Now we will construct a parse tree in the reverse of reduction.

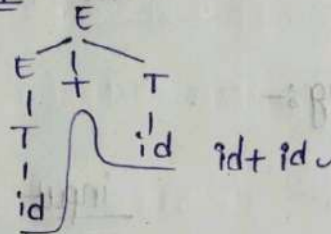
Step 1:-



Step 2:-



Step 3:-



Example 5:-

$S \rightarrow AA$
 $A \rightarrow aA$
 $A \rightarrow b$

Construct a parse tree for the input string "abb", using LR(0) parser?

sol:- Step 1: Augmented grammar:

$S' \rightarrow S$

$S \rightarrow AA$

$A \rightarrow aA$

$A \rightarrow b$

Step 2: Find LR(0) items:

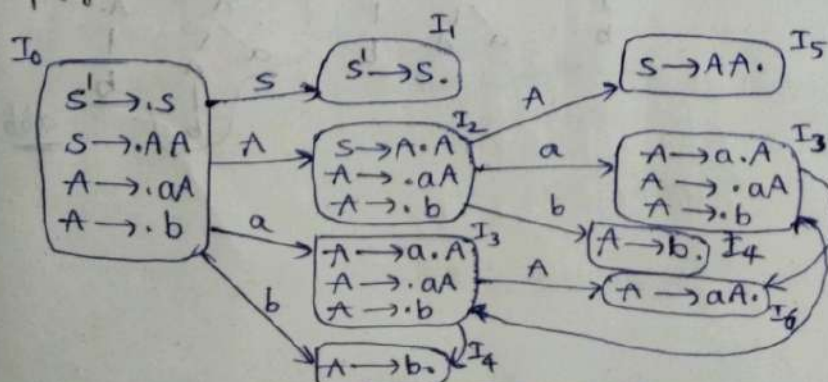
$S' \rightarrow \cdot S$

$S \rightarrow \cdot AA$

$A \rightarrow \cdot aA$

$A \rightarrow \cdot b$

Step 3:



Step 4: (RCO) table:

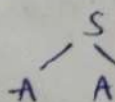
	Action			Goto	
	a	b	\$	S	A
0	S3	S4		1	2
1			Accept		
2	S3	S4			5
3	S3	S4			6
4	A→b	A→b	A→b		
5	S→AA	S→AA	S→AA		
6	A→aA	A→aA	A→aA		

step 5: Parsing:-

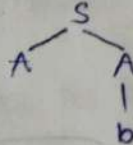
Stack	input	Action
\$0	a b b \$	shift a, 3
\$0a3	b b b \$	shift a, 4
\$0a3b4	b b \$	A→b; pop(b, 4), push A & state change
\$0a3A6	b b \$	A→aA; pop(a3A6), push A & state change
\$0A2	b b \$	shift b, 4
\$0A2b4	b b \$	A→b; pop(b, 4), push A & state change
\$0A2A5	b b \$	S→AA; pop(A2A5), push S & state change
\$0S1	b b \$	Accept

Parse tree:-

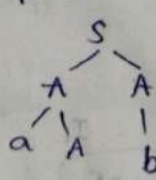
step 1:



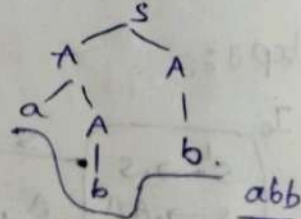
step 2:



step 3:

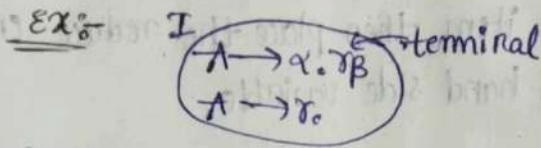


step 4:

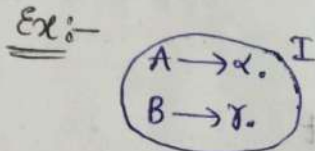


Parser Conflicts in LR(0):

1. SR Conflict: If any state in DFA contains both shift and reduce options then it is SR conflict.

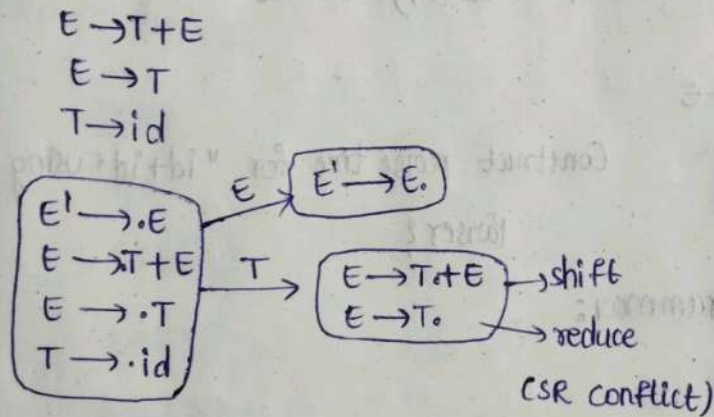


2. RR Conflict: If the same state contains more than one final item then it is RR conflict.



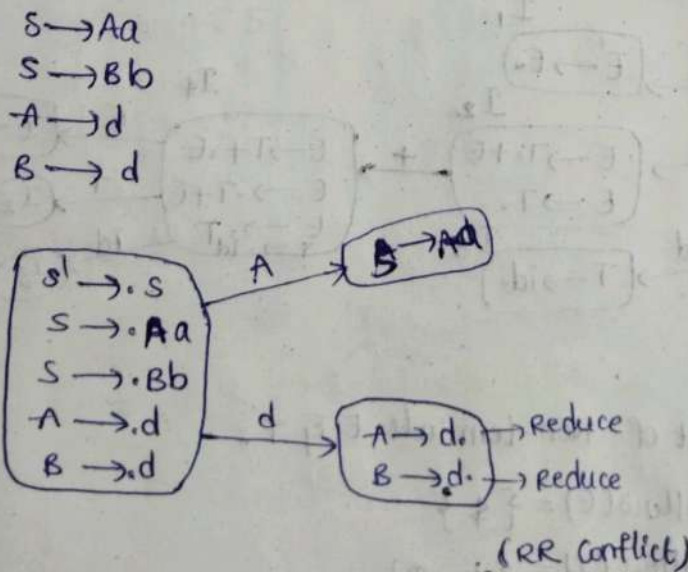
$\Rightarrow$ If no 2 types of conflicts it is LR(0) if it is LR(0) it is also SLR(0).

Example 6:-



$\therefore$ The given grammar is not LR(0).

Example 7:-



$\therefore$ The given grammar is not LR(0).

SLR(1) Parser :- Simple LR(1)

- ⇒ The procedure for constructing the parse table is similar to LR(0) Parser but there is one restriction in reducing the entries.
- ⇒ Whenever there is a final item then place the reduce entries under the follow symbols of left hand side variable.
- ⇒ If the SLR(1) parse table does not contain multiple entries that is free from conflicts is SLR(1) Grammar.

Rules:

- 1) If $S' \rightarrow S$. then $Action(I, \$) = Accept$
- 2) If $A \rightarrow \alpha \cdot a \beta$ then $Action(I, a) = shift\ j$
- 3) If $A \rightarrow \alpha \cdot B \beta$ then $Goto(I, B) = j$
- 4) If $A \rightarrow \alpha \cdot$ then $Action(I, Follow(A)) = Reduce\ A \rightarrow \alpha$

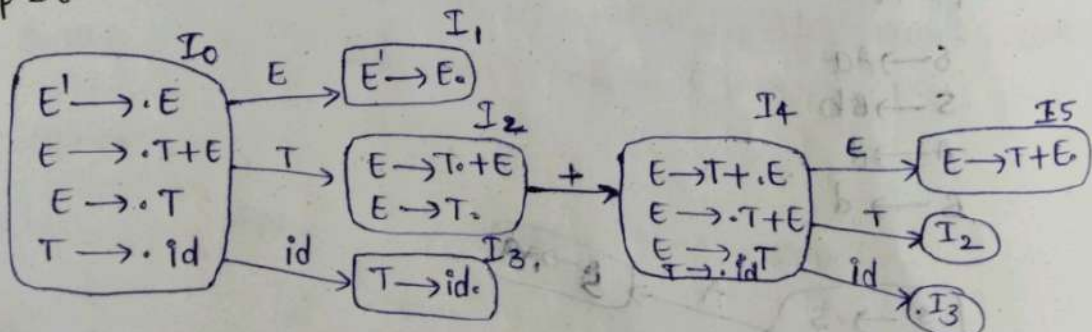
Example 8 :- $E \rightarrow T + E$
 $E \rightarrow T$
 $T \rightarrow id$

Construct parse tree for "id+id" using SLR(1) Parser?

Sol:- step 1: Augmented grammar:

$E' \rightarrow E$
 $E \rightarrow T + E$
 $E \rightarrow T$
 $T \rightarrow id$

step 2:



step 3:

finding follow set of Non-terminals E & T :

$Follow(E) = \{ \$ \}$

$Follow(T) = \{ +, \$ \}$

SLR(1) Table :

	Action			Goto	
	id	+	\$	E	T
0	S3			1	2
1			Accept		
2		S4	$E \rightarrow T$		
3		$T \rightarrow id$	$T \rightarrow id$		
4	S3			5	2
5			$E \rightarrow T+E$		

⇒ There are no multiple entries in Table. Hence it is SLR(1) Grammar.

Step 4: Parsing:

Stack

Input

Action

\$0

id+id\$

shift id, 3

\$0id3

+id\$

$T \rightarrow id$; pop id, 3, push T & state change

\$0T2

+id\$

shift +, 4

\$0T2+4

id\$

shift id, 3

\$0T2+4id3

\$

$T \rightarrow id$; pop id, 3, push T & state change

\$0T2+4T2

\$

$E \rightarrow T$; pop T2, push E & state change

\$0T2+4E5

\$

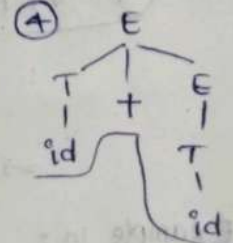
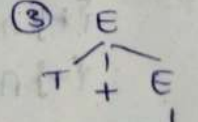
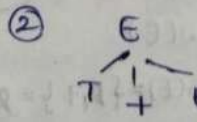
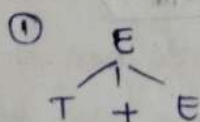
$E \rightarrow T+E$; pop T2+4E5, push E & state change

\$0E1

\$

Accept

Parse Tree :-



1) Shift reduce (SR) Conflict :-

$A \rightarrow \alpha \cdot a\beta \rightarrow \text{shift}$
 $B \rightarrow \alpha \cdot _ \rightarrow \text{reduce}$

SR Conflict in LR(0)

$\beta \rightarrow \alpha$ reduced in $\text{follow}(\beta)$

$\Rightarrow$ If $\{a\} \cap \text{follow}(\beta) \neq \emptyset$ then it is SR conflict in $\text{SLR}(1)$.

$\Rightarrow$ If $\{a\} \cap \text{follow}(\beta) = \emptyset$ then it is SR Conflict in LR(0) & not shift

2) RR Conflict :-

$$\begin{array}{l} A \rightarrow \alpha. \\ B \rightarrow \gamma. \end{array}$$

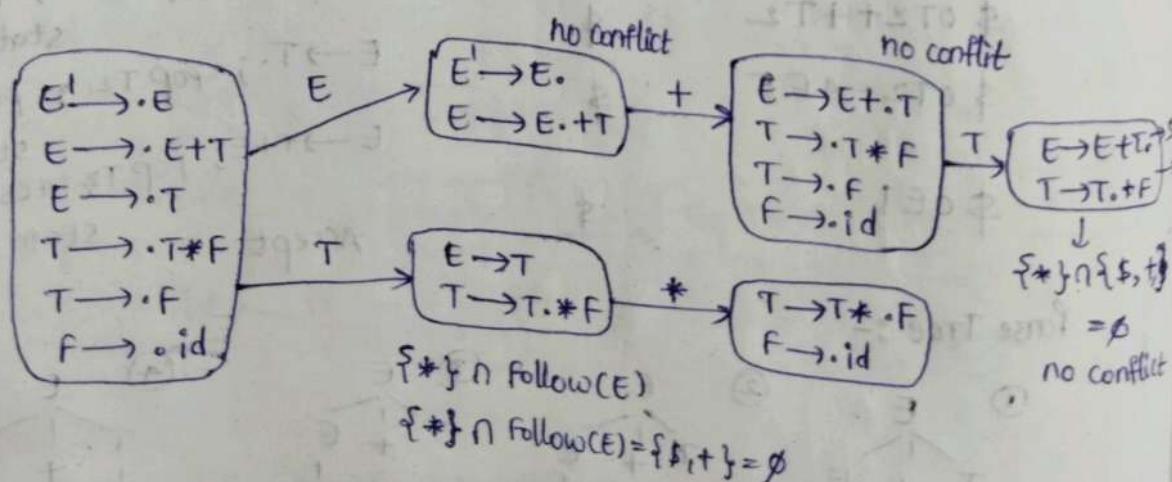
RR Conflict on $UR(o)$

$\Rightarrow$ If $\text{Follow}(A) \cap \text{Follow}(B) \neq \emptyset$ then it is RR conflict in $\text{SLR}(1)$.

⇒ If $\text{Follow}(A) \cap \text{Follow}(B) = \emptyset$ then it is RR Conflict. It is LR(0) but not in SLR(1).

Check for SLR(1):

Example 9: $E \rightarrow E + T$

$$E \rightarrow T$$
$$T \rightarrow T * F$$
$$T \rightarrow F$$
$$F \rightarrow \text{id}$$
$$\text{follow}(E) = \{ \$, + \}$$
$$\text{follow}(T) = \{\$, +, \cdot\}$$


$\Rightarrow$ It is SLR(1) but not LR(0)

Example 10: $S \rightarrow L = R$

$$S \rightarrow R$$
$$L \rightarrow *R$$
$$L \rightarrow id$$
 $R \rightarrow L$

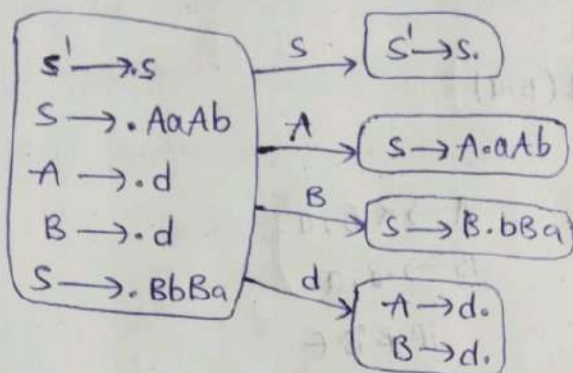
Not SLR (1) & not LR(0)

Example 11 : $S \rightarrow AaAb$

$S \rightarrow BbBa$

$A \rightarrow d$

$B \rightarrow d$



$\text{Follow}(A) \cap \text{Follow}(B)$

$\{a, b\} \cap \{a, b\} \neq \emptyset$

$\Rightarrow$ It has RR conflict

	a	b
$A \rightarrow d$	$A \rightarrow d$	$A \rightarrow d$
$B \rightarrow d$	$B \rightarrow d$	$B \rightarrow d$
RR	RR	RR

$\therefore$ The given grammar is not LR(0) not SLR(1).

LR(1) Items :

$\Rightarrow$ Mathematically, LR(1) item is the combination of LR(0) item and look ahead symbol.

* LR(1) item = LR(0) item + Look ahead

Ex:- $A \rightarrow \alpha.B\beta, \gamma$

$\Rightarrow \alpha.B\beta$ is LR(0) item

$\Rightarrow \gamma$ is a look ahead symbol.

* the look ahead is used to determine that where we place the final item.

Rules to find LR(1) items :

1) start with augmented production

$s' \rightarrow .s, \$$

Here $s' \rightarrow .s$ means we are expecting a derivate with s . When (s) dot reaches to end of s means derivation with s is completed only the $\$$ symbol is left.

2) Finding the closure

If $A \rightarrow \alpha \cdot B \beta, a$ add the production with non-terminal $B (B \rightarrow \cdot \gamma)$
lookahead for this B production rule is $\text{first}(\beta, a)$.

$$\begin{array}{l} A \rightarrow \alpha \cdot B \beta, a \\ B \rightarrow \cdot \gamma, \text{first}(\beta, a) \end{array}$$

$$\begin{array}{l} A \rightarrow \alpha \cdot B \beta, a \\ B \rightarrow \cdot \gamma, c/d \end{array}$$

if $\text{first}(\beta, a)$ is c/d

$$\begin{array}{l} A \rightarrow \alpha \cdot B, a \\ B \rightarrow \cdot \gamma, a \end{array}$$

if B is ϵ .

$\Rightarrow$ If β is ϵ then simply add lookahead symbol a for production on rule $B \rightarrow \cdot \gamma$.

ex:-

$$S' \rightarrow \cdot S, \$$$

$$S \rightarrow \cdot Aa, \$$$

$$A \rightarrow \cdot Bb, a$$

$$B \rightarrow \cdot d, b$$

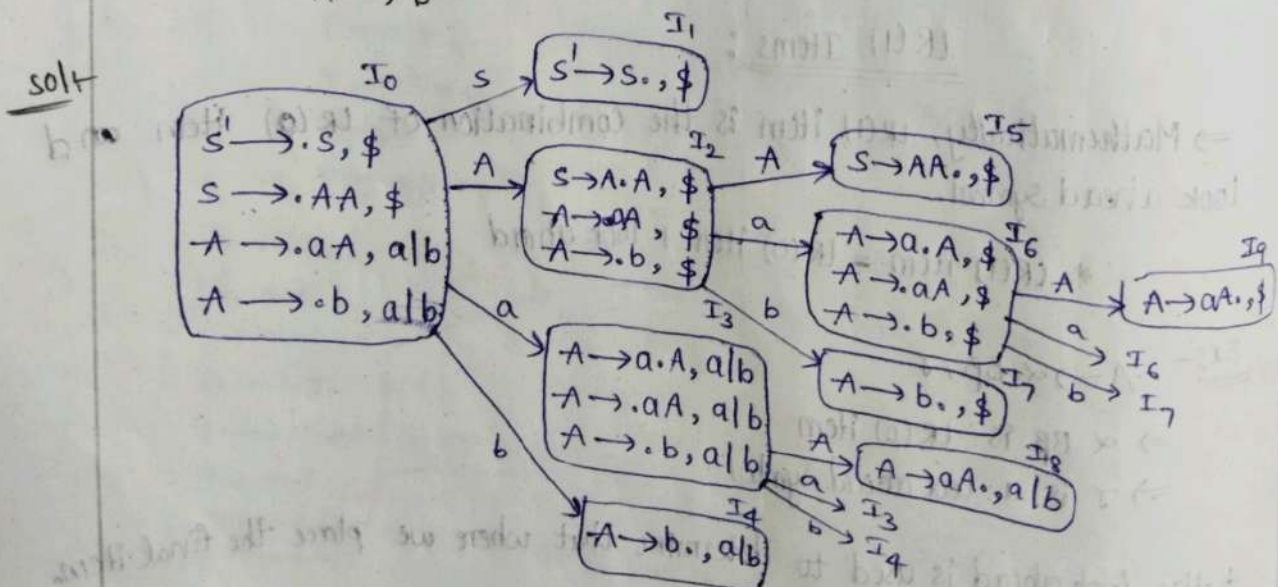
3) find goto

Example 12: Find LR(1) states for the following grammar.

$$S \rightarrow AA$$

$$A \rightarrow aA$$

$$A \rightarrow b$$



$\Rightarrow$ 10 states.

CLR(1) Parser :

Canonical LR(1) Parser :

- ⇒ In order to remove the invalid reduction actions, each state need to carry on additional information along with the previous information.
- ⇒ The additional information is referred as "look a head".
- ⇒ CLR(1) Parsers uses LR(1) item for the CFG.

$S \rightarrow AA$
 $A \rightarrow aA$
 $A \rightarrow b$

	Action			Goto	
	a	b	\$	S	A
0	S3	S4		1	2
1			Accept		
2	S6	S7			5
3	S3	S4			8
4	$A \rightarrow b$	$A \rightarrow b$			
5			$S \rightarrow AA$		
6	S6	S7			9
7		$A \rightarrow b$			
8	$A \rightarrow aA$	$A \rightarrow aA$			
9			$A \rightarrow aA$		

⇒ $A \rightarrow \alpha.$, a Action (1, a) = Reduce $A \rightarrow \alpha$

⇒ It is LR(1) Grammar & accepted by CLR(1).

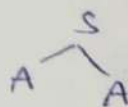
CLR(1) Parsing algorithm :

Stack	Input	Action
\$0	abb\$	S3
\$0a3	bb\$	S4
\$0a3b4	b\$	$A \rightarrow b$; pop(b4) push A, state change
\$0a3A8	b\$	$A \rightarrow aA$; pop(a3A8) push A, state change
\$0A2	b\$	S7

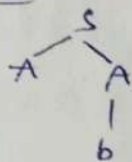
Stack	Input	Action
\$ 0A2b1	\$	$A \rightarrow b$; pop(b1), push A & state change
\$ 0A2A5	\$	$S \rightarrow AA$; pop(A2A5), push S & state change
\$ 0 S1	\$	Accept

Parse tree is:-

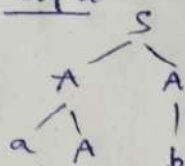
step 1:-



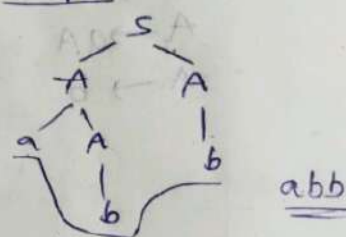
step 2:-



step 3:-



step 4:-



Parser Conflicts in LR(1):-

1) Shift Reduce (SR) Conflict:

$$\text{If } A \rightarrow \alpha \cdot a\beta, b \\ B \rightarrow \alpha \cdot, a$$

both are in same state

$$\boxed{\begin{matrix} A \rightarrow \alpha \cdot a\beta, b \\ B \rightarrow \alpha \cdot, a \end{matrix}} \begin{matrix} \rightarrow \text{shift} \\ \rightarrow \text{Reduce} \end{matrix}$$

shift symbol $\cap$ look ahead of Reduce production $\neq \emptyset$
 $\{a\} \cap \{a\}$

$\Rightarrow$ we have shift reduce conflict

2) Reduce-Reduce (RR) Conflict:

$$\text{If } \boxed{\begin{matrix} A \rightarrow \alpha \cdot, a \\ B \rightarrow \alpha \cdot, a \end{matrix}} \begin{matrix} \rightarrow \text{reduce} \\ \rightarrow \text{reduce} \end{matrix}$$

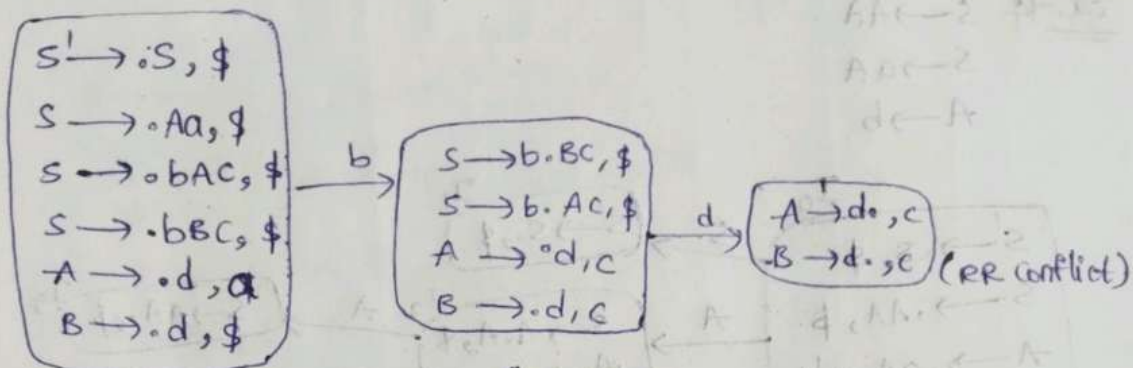
look ahead symbol $(RP_1 \cap RP_2) \neq \emptyset$

$\Rightarrow$ we have RR Conflict

Example 13:- check the following grammar LR(1) or not.

$$\begin{aligned} S &\rightarrow Aa \\ S &\rightarrow bAc \\ S &\rightarrow bBc \\ A &\rightarrow d \\ B &\rightarrow d \end{aligned}$$

solt



$\{c\} \cap \{c\} \neq \emptyset$
 not CLR(1) Grammar

LALR(1) Parser :-

It is very similar to CLR parsing, except the fact that we need to find out the states having CORE and combine as single state.

CORE :

core can be defined as the situation where two or more states have same LR(0) items I_i having different lookaheads.

Construction procedure of LALR(1) Parsing table :-

step 1: Write down the augmented grammar for the given grammar G .

step 2: Construct LR(0) items, say $C = \{I_0, I_1, \dots, I_n\}$

step 3: Search all the states of C and find out whether there exists any core, if found then combine them into one state by taking their union.

Assume the revised collection of LR(0) items is $C' = \{I_0, I_1, \dots, I_n\}$

step 4: Now develop the transition diagram.

step 5: The parsing actions can be performed as per the following rules.

S.1: If $A \rightarrow \alpha \cdot, a$ is in I_i , (then set $\text{action}(i, a) = \text{Reduce}$) and $\text{goto}[I_i, a] = I_k$ then set $\text{action}[i, a] = \text{shift } k$

S.2: If $A \rightarrow \alpha \cdot, a$ is in I_i , then set $\text{action}[i, a] = \text{Reduce } A \rightarrow \alpha (A \neq S')$

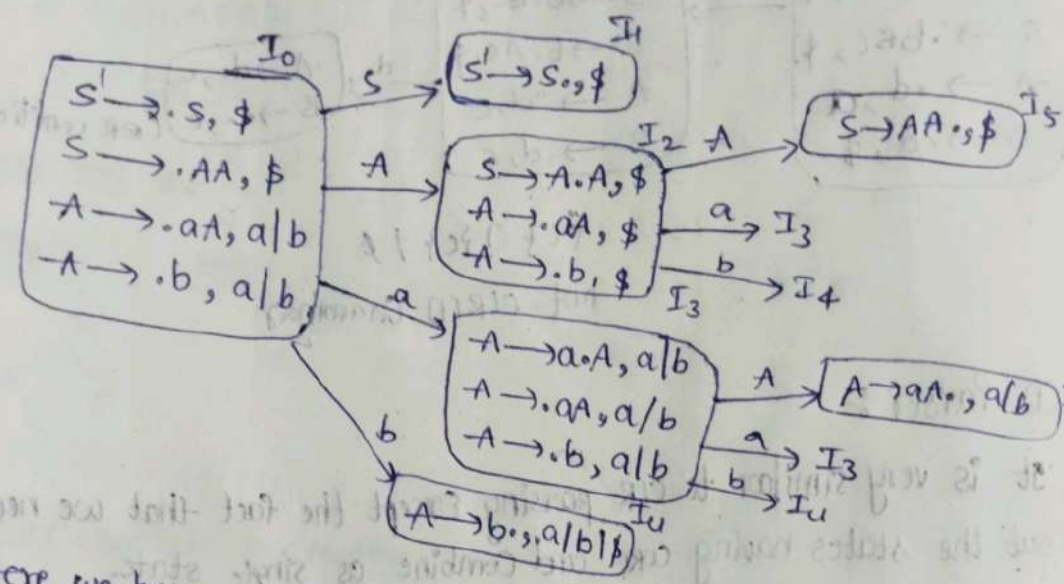
S.3: If $S' \rightarrow \cdot S, \$$ is in I_i , then set $\text{action}[i, \$] = \text{accept}$.

step 6: The goto transition for state j can be determined as follows:

If $\text{Goto}[I_i, A] = I_k$ then set $\text{goto}[i, A] = k$

step 7: Any entries which are not covered in step 5 and step 6 are treated as errors.

$S \rightarrow AA$
 $S \rightarrow aA$
 $A \rightarrow b$



Here we have 7 states

LR(0) Parse table :-

	Action			Goto	
	a	b	\$	S	A
0	s3	s4		1	2
1			Accept		
2	s3	s4			5
3	s3	s4			6
4	A -> b	A -> b	$A \rightarrow b$		
5			$S \rightarrow AA$		
6	A -> aA	A -> aA			

⇒ No multiple entries. So it is LR(0) Grammar.

Example 15 :-
 $S \rightarrow Aa$
 $S \rightarrow bAc$
 $S \rightarrow Bc$
 $S \rightarrow bBa$
 $A \rightarrow d$
 $B \rightarrow d$

Diagram illustrating LR(0) items and transitions for a grammar:

Grammar Rules:

- $S' \rightarrow S, \$$
- $S \rightarrow Aa, \$$
- $S \rightarrow bAC, \$$
- $S \rightarrow BC, \$$
- $S \rightarrow bBa, \$$
- $A \rightarrow d, a$
- $B \rightarrow d, c$

LR(0) Item Sets and Transitions:

- I_0 (Start State):**
 - $S' \rightarrow S, \$$
 - $S \rightarrow \cdot Aa, \$$
 - $S \rightarrow \cdot bAC, \$$
 - $S \rightarrow \cdot BC, \$$
 - $S \rightarrow \cdot bBa, \$$
 - $A \rightarrow \cdot d, a$
 - $B \rightarrow \cdot d, c$
- I_1 :**
 - $S' \rightarrow S, \$$
- I_2 :**
 - $S \rightarrow A \cdot a, \$$
- I_3 :**
 - $S \rightarrow B \cdot c, \$$
- I_4 :**
 - $S \rightarrow b \cdot AC, \$$
 - $A \rightarrow \cdot d, c$
 - $S \rightarrow b \cdot Ba, \$$
 - $B \rightarrow \cdot d, a$
- I_5 :**
 - $A \rightarrow d \cdot, a$
 - $B \rightarrow d \cdot, c$
- I_6 :**
 - $A \rightarrow d \cdot, c$
 - $B \rightarrow d \cdot, a$

Transitions:

- $I_0 \xrightarrow{S} I_1$
- $I_0 \xrightarrow{A} I_2$
- $I_0 \xrightarrow{B} I_3$
- $I_0 \xrightarrow{b} I_4$
- $I_0 \xrightarrow{d} I_5$
- $I_4 \xrightarrow{d} I_6$
- $I_5 \xrightarrow{\text{merge}} I_6$

Reductions:

- From I_5 to I_6 (labeled "reduce")

	a	c
s	$A \rightarrow d$ $B \rightarrow d$	$A \rightarrow d$ $B \rightarrow d$

Note :- Every $UL(u)$ is also $CR(u)$ but not vice versa.

Hand-drawn LR(0) item set diagram for the grammar:

- States (Items):**
 - I_0 : $E' \rightarrow \cdot E, \$$; $E \rightarrow \cdot BB, \$$; $B \rightarrow \cdot CB/d, c/d$
 - I_1 : $E' \rightarrow E \cdot, \$$
 - I_2 : $E \rightarrow B \cdot B, \$$; $B \rightarrow \cdot CB/d, \$$
 - I_3 : $B \rightarrow c \cdot B, c/d$; $B \rightarrow \cdot CB/d, c/d$
 - I_4 : $B \rightarrow d \cdot, c/d$
 - I_5 : $E \rightarrow BB \cdot, \$$
 - I_6 : $B \rightarrow C \cdot B, \$$; $B \rightarrow \cdot CB/d, \$$
 - I_7 : $B \rightarrow d \cdot, \$$
 - I_8 : $B \rightarrow CB \cdot, c/d$
 - I_9 : $B \rightarrow CB \cdot, \$$
- Transitions:**
 - $I_0 \xrightarrow{E} I_1$
 - $I_0 \xrightarrow{B} I_2$
 - $I_0 \xrightarrow{c} I_3$
 - $I_0 \xrightarrow{d} I_4$
 - $I_2 \xrightarrow{B} I_5$
 - $I_2 \xrightarrow{c} I_3$
 - $I_2 \xrightarrow{d} I_7$
 - $I_3 \xrightarrow{c} I_3$ (self-loop)
 - $I_3 \xrightarrow{d} I_4$
 - $I_3 \xrightarrow{B} I_6$
 - $I_3 \xrightarrow{c} I_8$
 - $I_6 \xrightarrow{B} I_9$
 - $I_6 \xrightarrow{c} I_3$
 - $I_6 \xrightarrow{d} I_7$

state	Action			go to	
	c	d	\$	E	B
I_0	S3	S4		1	2
I_1			Accept		6
I_2	S6	S7			5
I_3	S3	S4			8
I_4	r3	r3			

	c	d	\$	E	B
I_5			r_1		
I_6	s_6	s_7			9
I_7			r_3		
I_8	r_2	r_2			
I_9			r_2		

LALR(1) Parse table :

State	Action		goto		
	c	d	\$	E	B
I_0	s_{36}	s_{u7}		1	2
I_1			accept		
I_2	s_{36}	s_{u7}			5
I_{36}	s_{36}	s_{u7}			8/9
I_{u7}	r_3	r_3	r_3		
I_5			r_1		
I_{89}	r_2	r_2	r_2		

⇒ In CLR(1) parse table some productions which have different look ahead value in DFA.

$I_{36} = I_3, I_6$ have same production with different look ahead value.

$I_{u7} = I_4, I_7$

$I_{89} = I_8, I_9$

⇒ If we merge this states then we will get LALR(1) Parse table

Note :-

- 1) In LR(0), reduce is in full row.
- 2) In SLR(1), reduce is in follow(P)
- 3) In CLR & LALR reduce is written only on look ahead.
- 4) SLR(1) is simpler CLR(1), LALR(1) are very expansive.
- 5) Among All LR parsers, CLR(1) is more powerful, it accepts more grammar.
- 6) If $SLR(1) \quad LALR(1) \quad LR(1)$
 $(n_1 = n_2) \leq n_3$ — states
- 7) If there is no shift reduce conflict in LR(1) state then there will be no SR Conflict in LALR(1).
- 8) Merging of LR(1) states into LALR(1) will cause RR conflicts.